

AI201 | Applications of AI Engineering

Applications of AI Engineering Summer 2026 (@ Section 2A | Thursday 5PM - 7PM PT)
Personal Member ID#: 151677

Need help? Post on our [class slack channel](#) or email us at support@codepath.org

Welcome

Getting Started

► Course Info

AI Prompting Guide

Course Progress

▼ Module 1

Week 1

Week 2

Week 3

Week 4

► Module 2

► Module 3

Overview

Tinker (Lab)

Show (Project)

Report



Reminder: Project 4 is due by **Monday, June 29th at 2:59AM EDT.**

Submit

Show What You Know: Provenance Guard

 ~9–11 hours total

Platforms where people share original creative work — writing, music, art — are facing a new challenge: how do you know whether what someone posted was made by them, or generated by AI and passed off as human? Not to police creativity, but to protect attribution, build trust, and give audiences the context they actually need.

In this project, you'll build **Provenance Guard**: a backend system that any creative sharing platform could plug into to classify submitted content, score confidence in that classification, surface a transparency label to users, and handle appeals from creators who believe they've been misclassified.

This is your most independent project so far. No planning template is provided — you'll design your system architecture from scratch before writing a single line of implementation code.

How It's Graded

*A detailed breakdown of graded features and points can be found on the course [grading](#) page — **29 points total (25 required + 4 stretch)**. Read it before you start.*

Deliverables & Format

A few format requirements that are easy to miss — read these before you build:

- **Transparency label** → **write it as text in your README**. Your README must include the verbatim text of all three label variants (high-confidence AI, high-confidence human, uncertain) as a quoted string or a markdown table. A screenshot or mockup is welcome as an extra, but the written text is what's required — a screenshot alone is not enough.
- **Architecture diagram** → **put it in `planning.md`**. Include it under an `## Architecture` section. ASCII art is perfectly fine (so is a committed Mermaid block or an embedded sketch). This is guidance for where your diagram lives, not a format rule.
- **No starter repo, no planning template**. You create your own GitHub repo and design your `planning.md` structure yourself. Keep all of your evidence — audit-log sample, rate-limit configuration and chosen limits, label variants, and appeal handling — in your **README and committed source code**, which is the canonical record graders rely on.

Goals

By completing this project, you will be able to:

- Design and implement a multi-signal AI content classification pipeline.
- Build confidence scoring that communicates uncertainty rather than forcing binary outputs.
- Create end-user transparency labels that surface AI verdicts clearly and fairly.

- Implement an appeals workflow for contested classifications.
- Add production safety infrastructure: rate limiting and structured audit logging.

✓ Features

Required Features

- ☐ **Content Submission Endpoint:** Build an API endpoint that accepts a piece of text-based content (a poem, a short story excerpt, a blog post) for attribution analysis. The endpoint must return a structured response including the attribution result, confidence score, and the transparency label text that would be shown to the user.
- ☐ **Multi-Signal Detection Pipeline:** Your detection pipeline must use at least 2 distinct signals to classify content. Single-signal detection is not acceptable. Your `planning.md` and README must explain what each signal captures and why you chose them.
- ☐ **Confidence Scoring with Uncertainty:** Your system must return a confidence score, not just a binary label. The score should reflect genuine uncertainty — a 0.51 confidence should produce a meaningfully different transparency label than a 0.95. Your README must explain how you approached this and how you tested whether your scores are meaningful.
- ☐ **Transparency Label:** Design and implement the label that would be displayed to a reader on the platform. It must communicate the attribution result in plain language and make the confidence level meaningful to a non-technical reader. Include a typed description of all three label variants (high-confidence AI, high-confidence human, uncertain) in your README — write out the exact text each one displays. You're welcome to include a screenshot or mockup as well, but the written description is what's required.
- ☐ **Appeals Workflow:** Implement a mechanism for creators to contest a classification. At minimum, an appeal must: capture the creator's reasoning, log the appeal alongside the original decision, and update the content's status to "under review." Automated re-classification is not required.
- ☐ **Rate Limiting:** Implement rate limiting on your submission endpoint. Your README must document the limits you chose and your reasoning for those specific values.
- ☐ **Audit Log:** Every attribution decision — including confidence score, signals used, and any appeals — must be captured in a structured audit log. Document the log in your README (or via the `GET /log` output) with at least 3 entries visible.

Stretch Features

Complete any of these for extra credit. Update your `planning.md` before starting each one, and document any feature you complete in your **README** (what you built and how it works), not just `planning.md`.

- ☐ **Ensemble detection:** Incorporate 3 or more detection signals with a documented weighting or voting approach.
- ☐ **Provenance certificate:** Design and implement a "verified human" credential that a creator can earn through an additional verification step, including how it's displayed on their content.

- ☐ **Analytics dashboard:** Build a simple view showing detection patterns, appeal rates, and one additional metric of your choosing.
- ☐ **Multi-modal support:** Extend the pipeline to handle a second content type (e.g., image descriptions or structured metadata) in addition to text.

Hints

- A false positive (labeling a human's work as AI-generated) is worse than a false negative on a writing platform. Does your confidence scoring and label design reflect that asymmetry anywhere?
- Your confidence score is a design decision before it's a technical one. Decide what you want 0.5 to mean to a user, then figure out how to achieve that — not the other way around.
- The transparency label is a UX problem as much as a technical one. Show your label to someone who hasn't seen your project and ask what they understand from it before you finalize it.
- Rate limiting numbers aren't arbitrary. Think about realistic usage on a writing platform: how often does a single creator submit work? How would an adversary try to flood the system? Document your reasoning.
- Your detection doesn't need to be perfect — perfect AI detection is an unsolved problem. The system should acknowledge uncertainty honestly and give creators a path to appeal. That's the real engineering challenge here.
- Before reaching for AI assistance, go back to your spec. If a feature is unclear, the answer is usually in a question you didn't think through during Milestone 2.

Tools and Setup

This project uses a free backend stack — no new accounts required beyond your existing Groq account.

Recommended stack

Component	Tool	Notes
API framework	Flask	Free, lightweight
Detection signal 1	Groq (<code>llama-3.3-70b-versatile</code>)	Free tier — same account as Projects 1–3
Detection signal 2	Stylometric heuristics	Pure Python, no external libraries needed
Rate limiting	<code>Flask-Limiter</code>	Free
Audit log	SQLite (built-in) or structured JSON	No additional setup

Getting started

- ☐ **Create a new GitHub repository** for your project. Name it something like `ai201-project4-provenance-guard`. Initialize it with a README.

- ☐ **Clone your repo and create a virtual environment:**

```
python -m venv .venv
source .venv/bin/activate           # Mac/Linux
source .venv/Scripts/activate      # Windows (Git Bash)
# or: .venv\Scripts\activate       # Windows (Command Prompt)
```

- ☐ **Add a `requirements.txt`** to your repo:

```
flask>=3.0.0
flask-limiter>=3.5.0
groq==0.15.0
python-dotenv==1.0.1
```

Install with `pip install -r requirements.txt`. SQLite is built into Python, so no separate package is needed.

- ☐ **Create a `.env` file** in your repo root. Add it to your `.gitignore` — never commit it.

```
GROQ_API_KEY=your_key_here
```

A note on detection signals:

Your system needs at least 2 distinct signals. "Distinct" means they capture genuinely different properties of the text — not two versions of the same approach. A strong default pairing:

- **LLM-based classification** (Groq): ask the model to assess whether text reads as human or AI-generated. Captures semantic and stylistic coherence holistically.
- **Stylometric heuristics**: measurable statistical properties that differ between human and AI writing — sentence length variance, type-token ratio (vocabulary diversity), punctuation density, or average sentence complexity. AI text tends to be more uniform; human writing is more variable. Computable in pure Python.

These two signals are genuinely independent: one is semantic, one is structural. That makes the combination more informative than either alone.

Milestone 1: Understand the System and Define Your Architecture

 ~30 min

Before touching any code, understand the problem deeply and make your core architectural decisions. Provenance Guard has seven distinct features that need to work together. If you start building without understanding how they connect — how a submission flows through detection to a transparency label, how an appeal flows through to the audit log — you'll end up with a system that works feature-by-feature but breaks at the seams.

- ☐ Read the required features list in full. Then write, in plain English, the path a single piece of text takes from submission to the label a user sees. Name every system component it touches and what each one does. This is your architecture narrative — you'll use it in your `planning.md` and README.
- ☐ Decide on your two detection signals before you write any code. For each signal, write: what property of the text it measures, why that property differs between human and AI writing, and what it can't capture (every signal has blind spots). If you can't describe the blind spot, you don't understand the signal yet.
- ☐ Think through the false positive problem: what happens when your system misclassifies a human writer's work? Trace that scenario through your system — how does the confidence score reflect the uncertainty, what does the label say, and how does the creator appeal? This scenario should inform your decisions in Milestone 2.
- ☐ Sketch (on paper or in a text file) your API surface: what endpoints do you need? What does each one accept and return? You're not writing code yet — you're defining the contract that all other code will implement.

- Turn your architecture narrative into a diagram. Draw the two main flows: **(1) submission flow** — `POST /submit` → signal 1 → signal 2 → confidence scoring → transparency label → audit log → response; **(2) appeal flow** — `POST /appeal` → status update → audit log → response. Label each arrow with what passes between components (raw text, signal score, combined score, label text). You'll include this diagram in your `planning.md` and use it as context when prompting AI tools to generate code.

Checkpoint

You can describe the path a submitted piece of text takes through your system from start to finish, naming every component. You have chosen 2 detection signals and can explain what each captures and what it misses. You have a rough list of the API endpoints you need to build. You have a diagram showing both the submission and appeal flows.

Milestone 2: Write Your Spec Before Any Code

~1–2 hours

Write your `planning.md` before any implementation. There is no template — design the document structure yourself. What matters is that you address the five questions below with real, specific answers. A vague answer to "how will you represent uncertainty?" will produce a vague implementation, and you won't discover that until you're trying to explain a 0.62 confidence score to a non-technical user.

This document is also your primary AI prompting tool. When you reach Milestones 3–5, you'll give sections of this spec — plus the architecture diagram — to an AI tool to generate implementation code. A vague spec produces vague code. A spec that answers "what does signal 1's output look like?" and "what thresholds separate my three label variants?" gives AI tools something concrete to implement against.

- ☐ Create `planning.md` in your repo root. Design the structure yourself. At minimum, your document must address these five questions with specific, implementation-ready answers:
 - **Detection signals:** What are your 2+ signals? What does each one measure? What does each signal's output look like (a score between 0–1? a binary flag?), and how will you combine them into a single confidence score?
 - **Uncertainty representation:** What does a confidence score of 0.6 mean to your system? How will you map raw signal outputs to a calibrated score? What threshold separates "likely AI" from "uncertain" from "likely human"?
 - **Transparency label design:** What exact text will the label show for a high-confidence AI result? A high-confidence human result? An uncertain result? Write out the three label variants now, before you build the UI.

- **Appeals workflow:** Who can submit an appeal? What information do they provide? What does the system do when an appeal is received — what status changes, what gets logged? What would a human reviewer see when they open the appeal queue?
- **Anticipated edge cases:** What types of content will your system handle poorly? Name at least two specific scenarios — not generic risks like "inaccurate detection," but specific cases like "a poem with heavy use of repetition and simple vocabulary that your heuristics might score as AI-generated."

- ☐ Add an `## Architecture` section to your `planning.md`. Include the diagram you drew in Milestone 1 (ASCII art is fine) and a 2–3 sentence narrative describing the submission and appeal flows. This section travels with you into Milestones 3–5 as the reference diagram for AI code generation.

- ☐ Add an `## AI Tool Plan` section to your `planning.md`. For each of the three implementation milestones, specify:
 - **M3 (submission endpoint + first signal):** Which spec sections you'll provide to the AI tool (hint: your detection signals section + the diagram), what you'll ask it to generate (Flask app skeleton + the first signal function), and how you'll verify the output (test with a few inputs directly before wiring into the endpoint).
 - **M4 (second signal + confidence scoring):** Which spec sections you'll provide (detection signals + uncertainty representation + diagram), what you'll ask for (second signal function + scoring logic), and what you'll check (do scores vary meaningfully between clearly AI and clearly human text?).
 - **M5 (production layer):** Which spec sections you'll provide (label variants + appeals workflow + diagram), what you'll ask for (label generation logic + the `/appeal` endpoint), and how you'll verify (test all three label variants are reachable and that an appeal updates status correctly).
- ☐ Review your label variants. Revise if needed before you start building.
- ☐ Update `planning.md` before starting any stretch features.

Checkpoint

`planning.md` addresses all five questions with specific answers. You have written out the three label variants (high-confidence AI, high-confidence human, uncertain). Your confidence scoring approach produces different labels at different score ranges — not a binary flip at 0.5. Your `## Architecture` section includes the diagram from Milestone 1. Your `## AI Tool Plan` section covers all three implementation milestones with specific sections, requests, and verification steps.

Milestone 3: Build the Submission Endpoint and First Detection Signal

~2–3 hours

Build your Flask app, implement the submission endpoint, and get your first detection signal working end-to-end. Don't build all the features at once! Get one signal producing a result you can inspect before adding the second. A single working signal is easier to debug than two broken ones.

New to Flask? Start with the [Flask Quickstart](#) — a 20-minute primer covering a minimal POST/GET app, `jsonify`, Flask-Limiter setup, and a simple log helper.

- ☐ Before writing any code, use your `planning.md` + architecture diagram to prompt an AI tool. Give it your detection signals section and the diagram and ask it to generate: (1) the Flask app skeleton with the `POST /submit` route stub, and (2) the first signal function. Review the output carefully — check that the function signature matches your spec's description of what the signal returns (a score? a binary flag?), and that the Flask route structure matches your API contract. Edit before using; don't paste blindly.
- ☐ Set up your Flask app. Create a `POST /submit` endpoint that accepts a JSON body with at minimum a `text` field and a `creator_id` field. For now, have it return a hardcoded response so you can verify the route works before adding any logic.
- ☐ Implement your first detection signal. If you're using Groq as your first signal, write a function that sends the text to the API with a prompt that returns a structured assessment. Test it independently before wiring it into the endpoint — call the function directly with a few test inputs and inspect the output.
- ☐ Wire the first signal into the submission endpoint. The endpoint should now return a response with at least: a `content_id` (a unique ID for this submission), the attribution result from signal 1, a placeholder confidence score, and a placeholder label. The `content_id` is essential — the appeal endpoint needs it, and it should appear in your `/submit` response and audit log.

Test with a curl command:

```
curl -s -X POST http://localhost:5000/submit \  
  -H "Content-Type: application/json" \  
  -d '{"text": "The sun dipped below the horizon, painting the sky in hues of
```

You should see a JSON response with `content_id`, `attribution`, `confidence`, and `label` fields. Save the `content_id` — you'll use it to test appeals in Milestone 5.

- ☐ Set up your audit log. Before moving on, every call to the submission endpoint should write a structured entry to the log — timestamp, content ID, attribution result, signal 1 score. You'll extend the log in Milestone 4; start simple and make it structured (JSON or SQLite, not `print()` statements). A well-structured entry looks like this:

```
{
  "content_id": "3f7a2b1e-...",
  "creator_id": "test-user-1",
  "timestamp": "2025-04-01T14:32:10.123Z",
  "attribution": "likely_ai",
  "confidence": 0.78,
  "llm_score": 0.81,
  "status": "classified"
}
```

- ☐ Add a `GET /log` endpoint that returns the most recent audit log entries as JSON. The project requires showing the audit log with at least 3 structured entries — without a `/log` endpoint (or equivalent), you'll have no clean way to surface this in your README. Keep it simple: `return jsonify({"entries": get_log()})`. In a real system this would require auth; here it's for documentation and grading visibility.

Checkpoint

Your Flask app runs. `POST /submit` returns a JSON response including `content_id`, attribution result, and a placeholder confidence score. Each submission writes a structured entry to the audit log. `GET /log` returns those entries as JSON. You can inspect the log and see your test submissions.

Milestone 4: Add the Second Signal and Implement Confidence Scoring

~2 hours

Add your second detection signal and implement real confidence scoring. This milestone is where your detection pipeline becomes multi-signal. The key engineering challenge is combining two signals — each with its own output format and reliability profile — into a single, calibrated confidence score.

- ☐ Use your detection signals section + uncertainty representation section + architecture diagram to prompt an AI tool. Ask it to generate: (1) the second signal function, and (2) the confidence scoring logic that combines both signals according to your spec. Verify that the generated scoring function actually matches the thresholds you defined in your planning document — AI tools sometimes implement reasonable-looking scoring that silently diverges from your specified ranges. If it does, correct it before wiring it in.
- ☐ Implement your second detection signal as a standalone function, tested independently before integration. If you're using stylometric heuristics, compute 2–3 specific metrics (e.g., sentence length variance, type-token ratio) and decide how to combine them into a single signal score. Test it on the same inputs you used for signal 1 — do the two signals agree? Disagree? Understanding where they diverge tells you something about each signal's strengths.
- ☐ Implement your confidence scoring logic. Both signals now produce a score — combine them according to your `planning.md` spec. Your combined score should: vary meaningfully across clearly different inputs (a highly polished, uniform paragraph vs. a casual, irregular piece should produce different scores), and map to at least 3 distinct label categories (e.g., "Likely AI-generated," "Uncertain," "Likely human-written").

- ☐ Test your scoring with at least 4 deliberately chosen inputs: something you're confident is AI-generated, something you're confident is human-written, and two borderline cases. Do the scores match your intuition? If not, investigate why before moving on — a miscalibrated scoring function will undermine everything built on top of it.

If you need test inputs, here's a starting set:

```
# Clearly AI-generated (should score high)
```

```
"Artificial intelligence represents a transformative paradigm shift in modern  
It is important to note that while the benefits of AI are numerous, it is equally  
essential to consider the ethical implications. Furthermore, stakeholders across  
various sectors must collaborate to ensure responsible deployment."
```

```
# Clearly human-written (should score low)
```

```
"ok so i finally tried that new ramen place downtown and honestly?"
```

underwhelming. the broth was fine but they put WAY too much sodium in it and i was thirsty for like three hours after. my friend got the spicy version and said it was better. probably won't go back unless someone drags me there"

Borderline: formal human writing (may score mid-high on stylometrics)

"The relationship between monetary policy and asset price inflation has been extensively studied in the literature. Central banks face a fundamental tension between their mandate for price stability and the unintended consequences of prolonged low interest rates on equity and real estate valuations."

Borderline: lightly edited AI output (should ideally score mid-range)

"I've been thinking a lot about remote work lately. There are genuine tradeoffs between flexibility and no commute on one side, isolation and blurred work-life boundaries on the other. Studies show productivity varies widely by individual and role."

If any of these produce scores that don't match your intuition, print both signal scores separately to find which one is misbehaving.

- ☐ Update your audit log to capture both signals' individual scores alongside the combined confidence score.

Checkpoint

Both detection signals are running and their outputs are combined into a single confidence score. Submitting clearly AI-generated text produces a noticeably different score than clearly human-written text. The audit log now records individual signal scores and the combined result. You have tested at least 4 inputs spanning the confidence range.

Milestone 5: Implement the Production Layer

 ~2–3 hours

Add the four production features that turn your detection pipeline into a real system: the transparency label, the appeals workflow, rate limiting, and a complete audit log. These features are independent enough to build in any order — but the transparency label and appeals workflow both depend on the confidence scoring from Milestone 4, so verify that first.

- ☐ Use your transparency label variants and appeals workflow sections from `planning.md` + the architecture diagram to prompt an AI tool. Ask it to generate: (1) a label generation function that maps confidence scores to the correct label text, and (2) the `POST /appeal` endpoint. Verify the label function against your spec's thresholds — ask it to produce all three variants and confirm the text matches what you wrote. For the appeal endpoint, check that it updates status and logs correctly before considering it done.
- ☐ **Transparency label:** Implement the three label variants you designed in `planning.md`. The label returned by the submission endpoint must change based on the confidence score — it should not be the same text regardless of score. Test that all three variants are reachable by submitting inputs that produce different confidence levels.
- ☐ **Appeals workflow:** Build a `POST /appeal` endpoint that accepts a `content_id` and `creator_reasoning` field. The endpoint should: update the content's status to "under review" in whatever storage you're using, log the appeal alongside the original classification decision in the audit log, and return a confirmation that the appeal was received. You do not need to implement automated re-classification.

To test, use the `content_id` from any earlier `/submit` response:

```
curl -s -X POST http://localhost:5000/appeal \
  -H "Content-Type: application/json" \
  -d '{"content_id": "PASTE-CONTENT-ID-HERE", "creator_reasoning": "I wrote th
```

Then verify the appeal is in the log with `GET /log` — the entry should show `"status": "under_review"` and the `appeal_reasoning` field populated.

- ☐ **Rate limiting:** Apply `Flask-Limiter` to your submission endpoint. Choose limits that reflect realistic usage (a writer submitting their own work) while preventing abuse (a script flooding the system). Document your chosen limits and reasoning in your README — the numbers should be defensible, not arbitrary.

⚙️ **Flask-Limiter setup note:** Flask-Limiter $\geq 3.x$ requires a `storage_uri` parameter. The simplest setup for local development uses in-memory storage:

```
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address

limiter = Limiter(
    get_remote_address,
    app=app,
    default_limits=[],
    storage_uri="memory://",
)
```

Apply the limit to your submit route:

```
@app.route("/submit", methods=["POST"])
@limiter.limit("10 per minute;100 per day")
def submit():
    ...
```

Without `storage_uri="memory://"` you may see a warning or error on startup.

To test that rate limiting is working, run this in a new terminal window while your Flask server is running (it sends 12 rapid requests — more than the 10/minute limit):

```
for i in $(seq 1 12); do
  curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:5000/submit \
    -H "Content-Type: application/json" \
    -d '{"text": "This is a test submission for rate limit testing purposes or
done
```

You should see `200` responses for the first 10 and `429` responses after. Capture those 429 responses in your README (paste the status-code output) — that's the evidence graders need.

- ☐ **Complete audit log:** Verify that your log now captures everything required: timestamp, content ID, attribution result, confidence score, both individual signal scores, and whether an appeal has been filed. Check that the format is structured — JSON or a formatted log file, not unformatted console output. Generate at least 3 entries so you have something to document in your README.

Checkpoint

All four production features are working: the transparency label varies by confidence level, appeals can be submitted and are reflected in the audit log, rate limiting triggers when the limit is exceeded, and the audit log has at least 3 structured entries covering submissions and at least one appeal. All of these work end-to-end without workarounds.

Milestone 6: Document and Walk Through Your Work

~1–2 hours

Write your README, then record a short portfolio walkthrough of your project. Your README is the canonical record of your work — it needs to explain your architecture and design decisions, not just list the features.

- ☐ Write your README covering all required sections from the submission checklist. The detection signal and confidence scoring sections should explain your reasoning, not just your implementation — why these signals, why this scoring approach, what you'd change if you were deploying this for real.
- ☐ In the confidence-scoring section, include **two example submissions with noticeably different confidence scores** — one high-confidence and one lower-confidence case — showing the actual scores (you can lift these from your Milestone 4 testing). This is what shows your scoring produces meaningful variation, not a constant.

- ☐ **Include a typed description of all three variants (high-confidence AI, high-confidence human, uncertain)** — write out the exact text each one displays. You're welcome to include a screenshot or mockup as well, but the written description is what's required.
- ☐ Write the known limitations section honestly. Name at least one specific type of content your system would likely get wrong and explain why — tied to a property of your signals, not a generic "it needs more data."
- ☒ ~~Write your spec reflection in the README: describe one way the spec helped guide your implementation and one way your implementation diverged from it and why.~~
- ☐ Add the AI usage section. Describe at least 2 specific instances: what you directed the AI to do, what it produced, and what you revised or overrode.
- ☐ Record a short portfolio walkthrough (a couple of minutes) of your project. Briefly show your system working end-to-end and talk through a few of your design decisions. Keep it short and unpolished — it's a great portfolio piece and good practice explaining your work to others. The detailed evidence (audit-log sample, rate-limit behavior, label variants, appeal handling) lives in your README; the walkthrough is just you giving a quick tour.

Checkpoint

README covers all required sections with substantive explanations of design decisions, not just feature descriptions. All three transparency label variants are written out. You've recorded a short portfolio walkthrough giving a quick tour of your system.

Submitting Your Project

Submit all of the following through the Course Portal:

- ☐ Link to your GitHub repository
- ☐ `planning.md` in your repo root (written before implementation, updated before any stretch features)

☐ `README.md` including:

- Architecture overview: the path a submission takes from input to transparency label
- Detection signals: what each signal measures, why you chose it, and what it misses
- Confidence scoring: how you combined signals into a score, how you validated it's meaningful, and **two example submissions with noticeably different confidence scores** (one high-confidence, one lower-confidence) showing the actual scores
- Transparency label: typed description of all three variants (high-confidence AI, human, uncertain) showing the exact text each one displays; screenshot or

mockup optional

- Rate limiting: the limits you chose and your reasoning for those specific values
- Known limitations: at least one specific type of content your system would likely misclassify and why
- Spec reflection: one way the spec helped you, one way implementation diverged from it and why
- AI usage section: at least 2 specific instances describing what you directed the AI to do and what you revised or overrode

- ❑ A short portfolio walkthrough video (a couple of minutes): a quick tour of your project where you show it working end-to-end and talk through a few design decisions. Keep it short and unpolished — it's a portfolio piece and good practice explaining your work. The detailed evidence lives in your README and source code; the walkthrough is just your tour of it.



Resources

- [RepairSafe lab starter \(Lab 4\)](#): The Week 4 lab repo — a worked example of an AI classification + safety-label backend in the same problem space as this project

- [AI Prompting Guide](#): Patterns for directing AI tools to debug, design, and refine your code
- [Flask Quickstart](#): A 20-minute primer on a minimal POST/GET Flask app, `jsonify`, Flask-Limiter setup, and a simple log helper
- [Study Hall](#): Bring your architecture sketch and questions — don't struggle alone (see the Study Hall schedule and recordings)

